ARCHITECTURE AND WORKFLOW

Knowledge Handover Platform

All 13 diagrams from your architecture and workflow files, rendered. Parse errors in the source are fixed; the list is at the bottom.

- [ ] Start and end points
- [ ] Human review and validation gate
- [ ] Data sources and storage
- [ ] Application logic and services

ARCHITECTURE

A1 High-level system architecture
A2 Component architecture
A3 Data flow
A4 Knowledge graph schema
A5 Deployment architecture
A6 Connector architecture

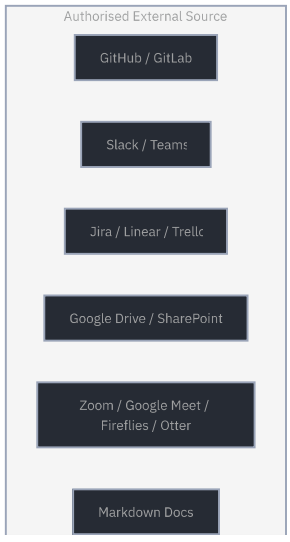
WORKFLOW

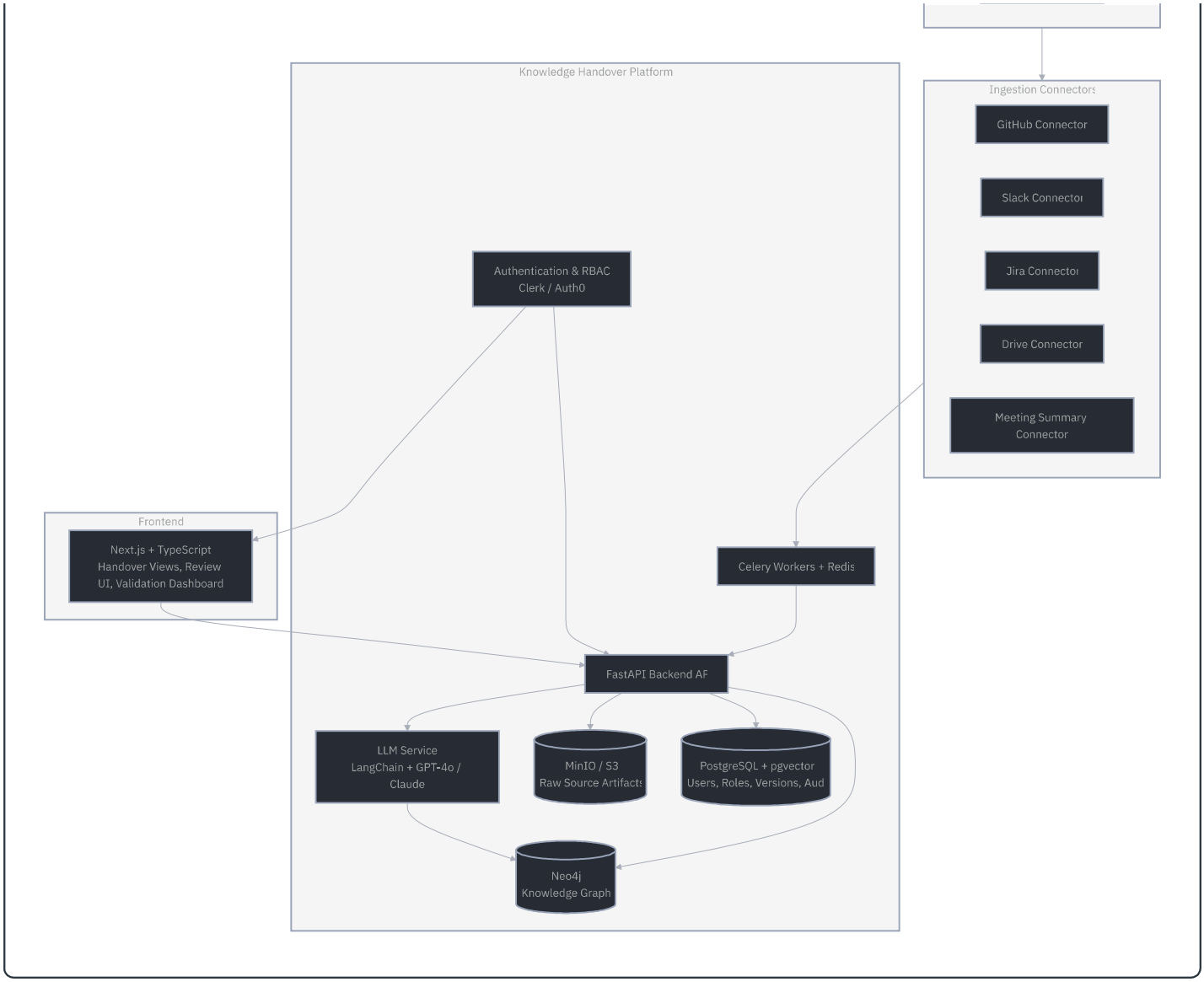
W1 Core handover workflow
W2 Detailed workflow
W3 Sequence
W4 Sequence
W5 Sequence
W6 Validation gate decision flow
W7 Phase 3

Architecture

A1 High-level system architecture

Sources feed the connectors and workers, which write through the API to the graph, database and object store. The frontend and auth sit on top.

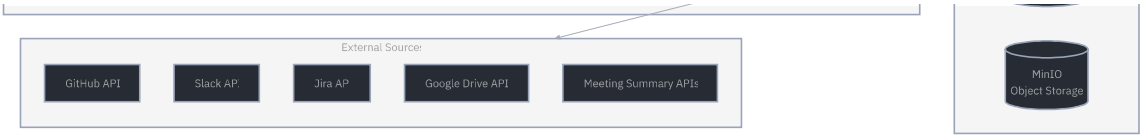




A2 Component architecture

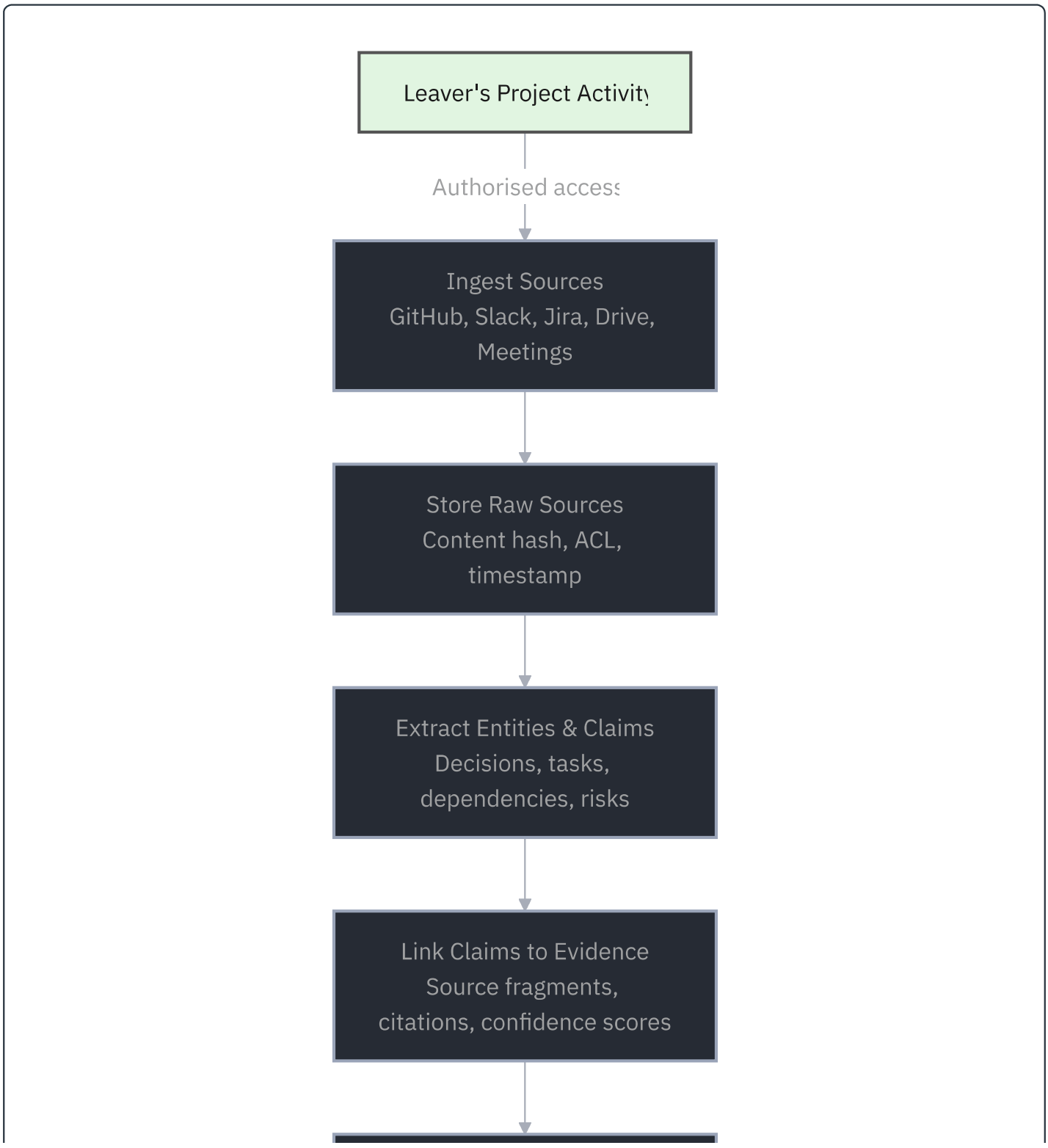
Services behind the API gateway, the data stores they use, and the AI and external layers they depend on.

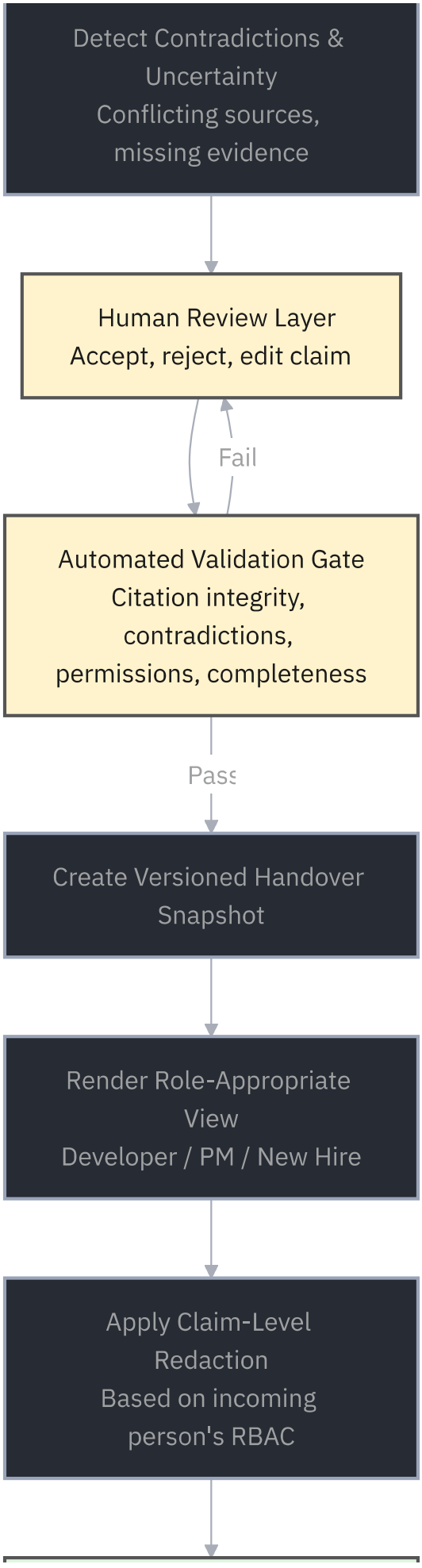




A3 Data flow, connect to handover

From the leaver's project activity to a shared, acknowledged handover. A failed validation sends the draft back to review.

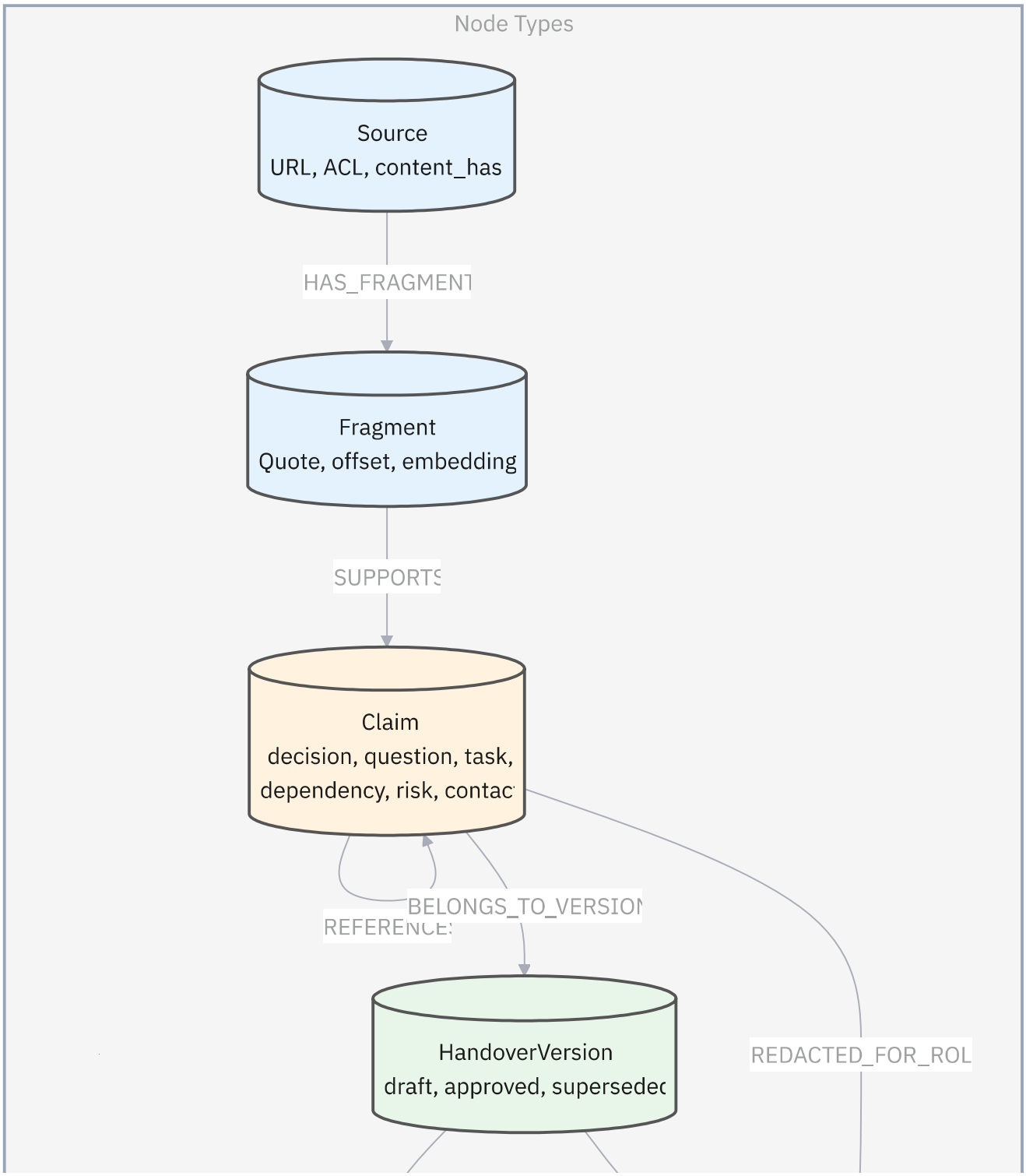


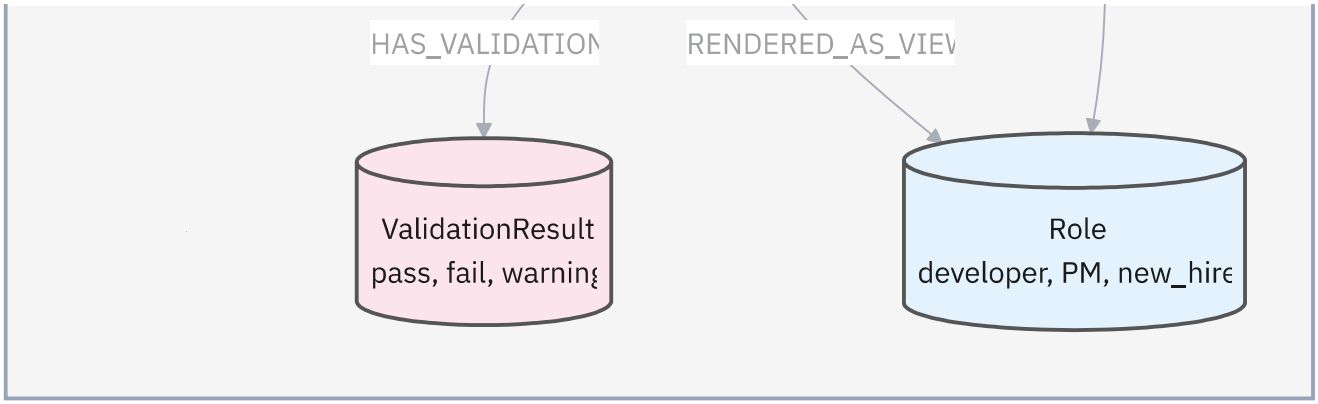

Detect Contradictions & Uncertainty
Conflicting sources, missing evidence
Human Review Layer
Accept, reject, edit claim
Fail
Automated Validation Gate
Citation integrity, contradictions, permissions, completeness
Pass
Create Versioned Handover Snapshot
Render Role-Appropriate View
Developer / PM / New Hire
Apply Claim-Level Redaction
Based on incoming person's RBAC

Share Handover + Track Acknowledgement

A4 Knowledge graph schema

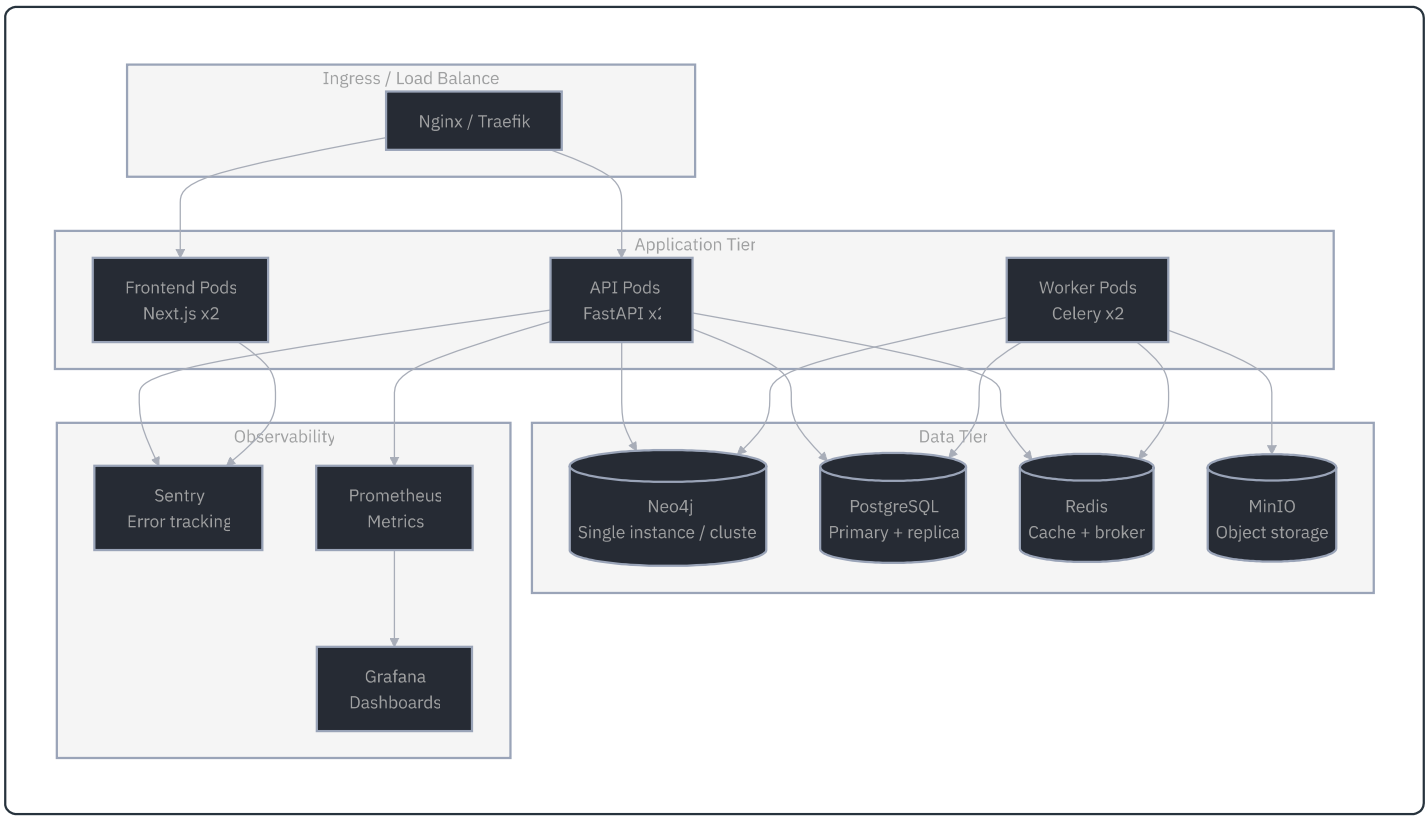
Node types and relationship types stored in Neo4j.





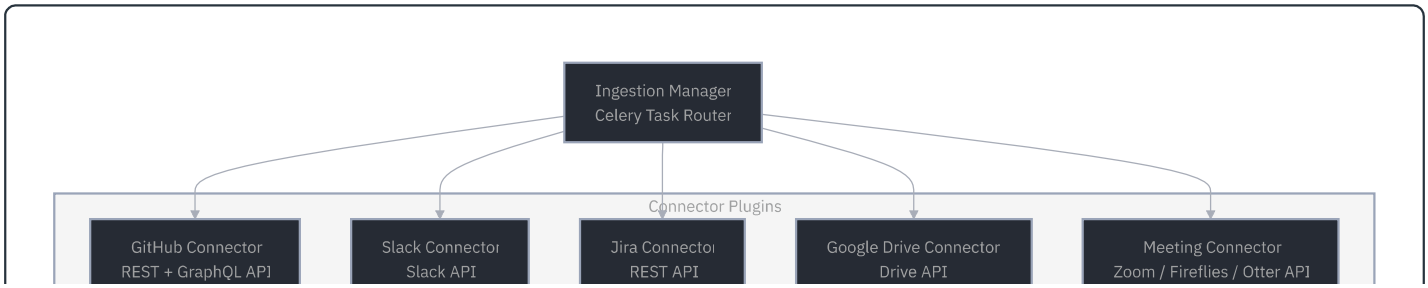
A5 Deployment architecture

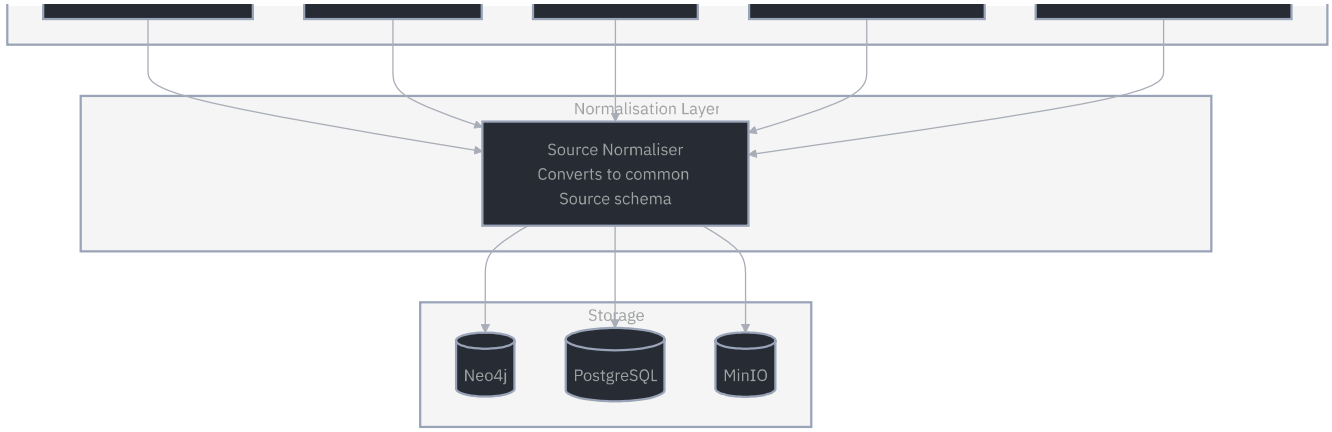
Ingress, application pods, data tier and observability, for Docker Compose or Kubernetes.



A6 Connector architecture

Each pluggable connector converts its source into one common schema before anything is stored.





Workflow

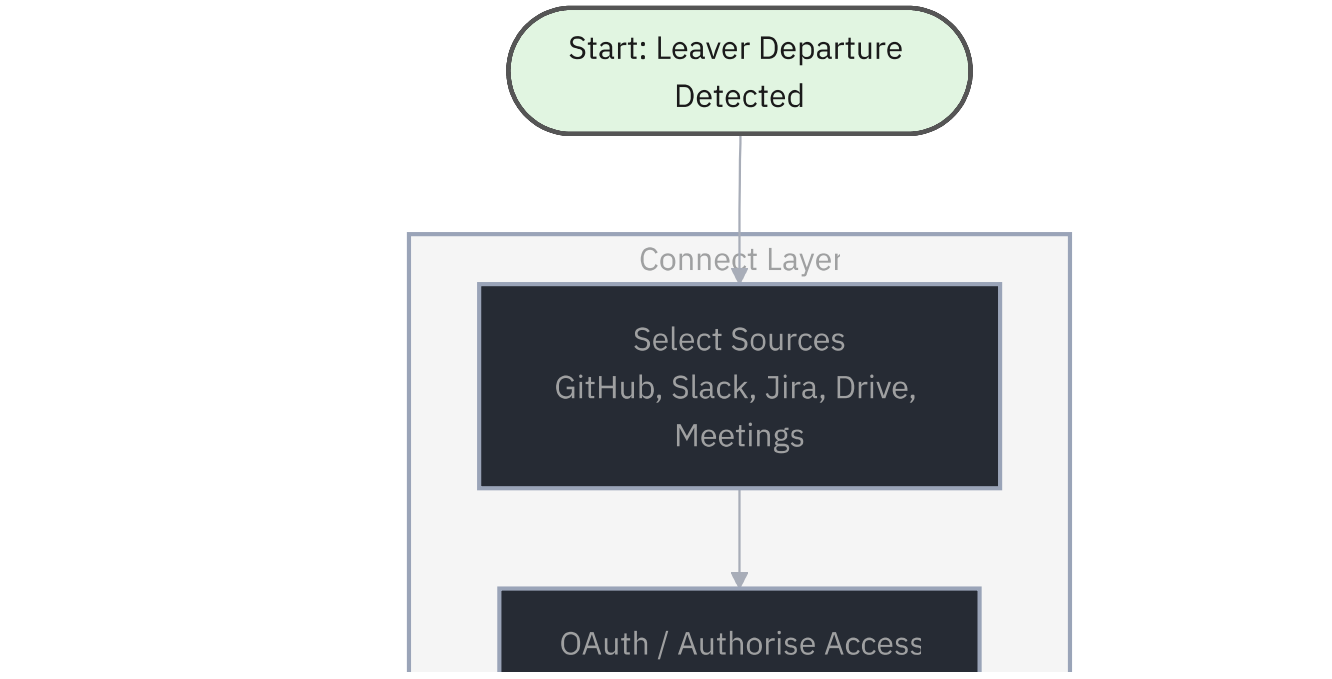
W1 Core handover workflow

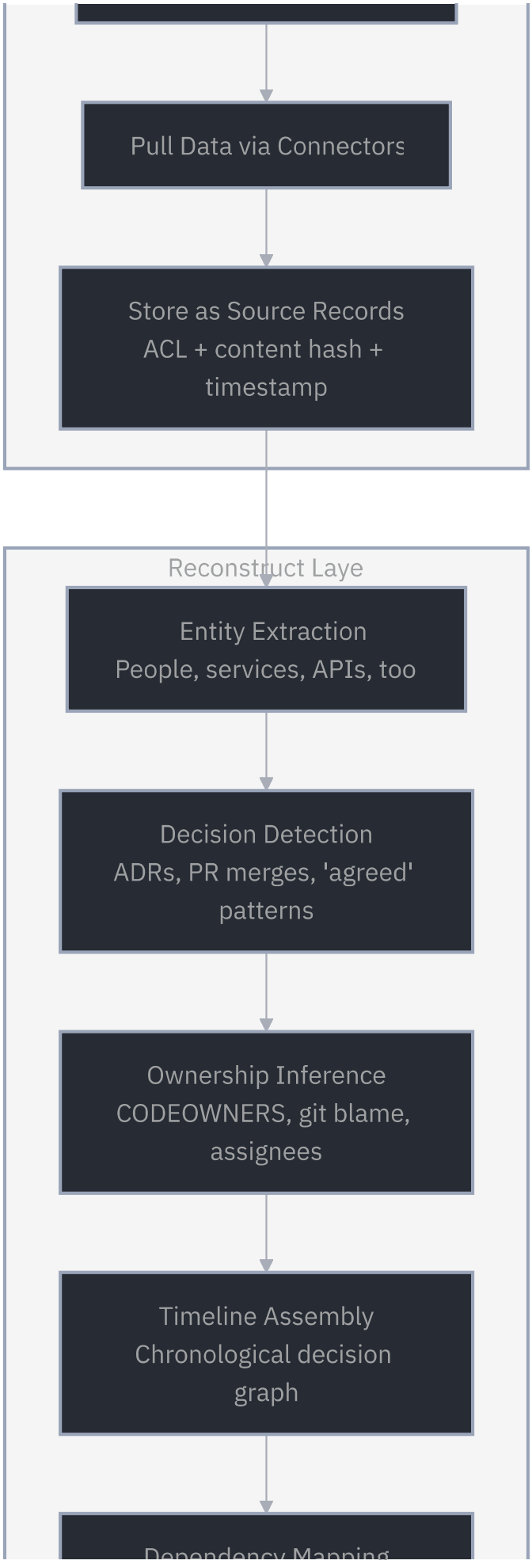
The six layers in order. Review and Validate are the two gates.

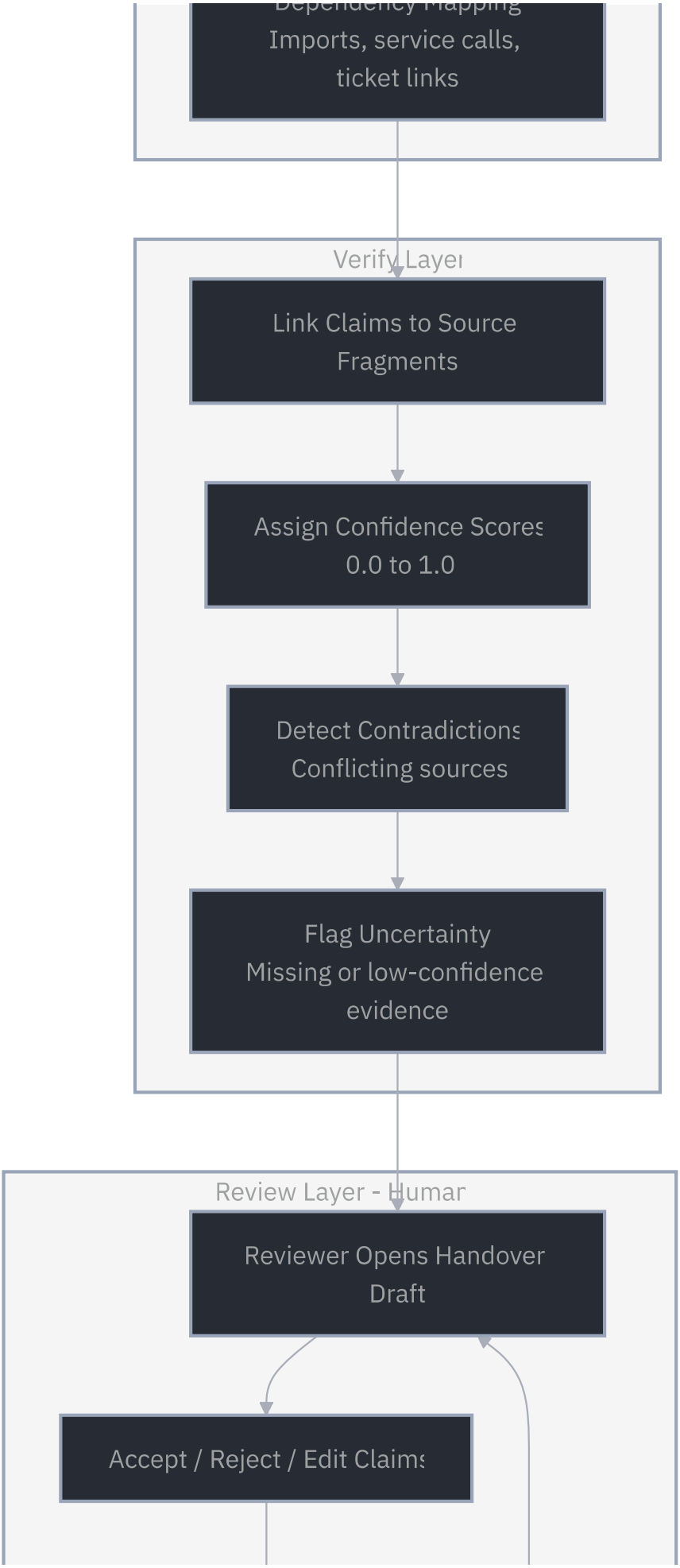


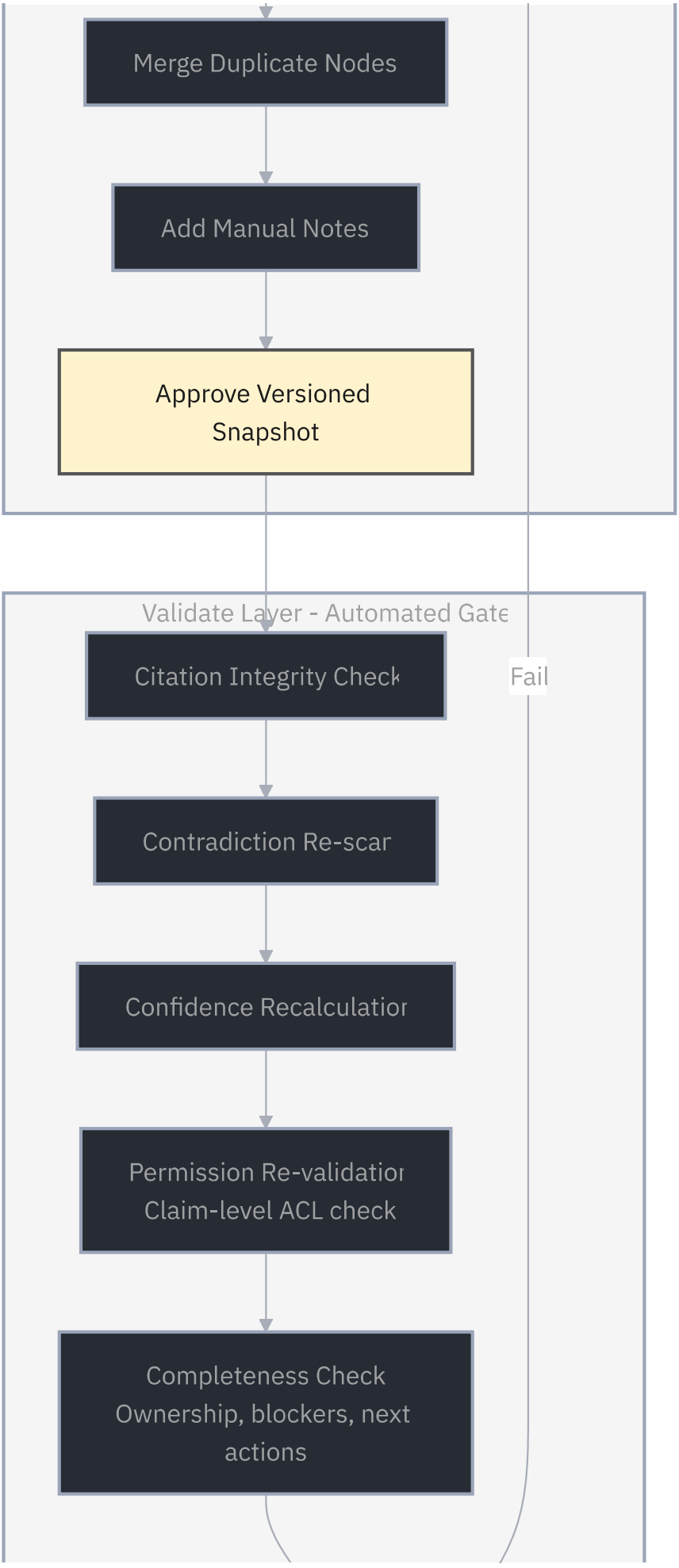
W2 Detailed workflow, source to handover

Every step in the six layers. A failed gate returns the draft to the reviewer.




Pull Data via Connectors
Store as Source Records
ACL + content hash +
timestamp
Reconstruct Laye
Entity Extraction
People, services, APIs, too
Decision Detection
ADRs, PR merges, 'agreed'
patterns
Ownership Inference
CODEOWNERS, git blame,
assignees
Timeline Assembly
Chronological decision
graph

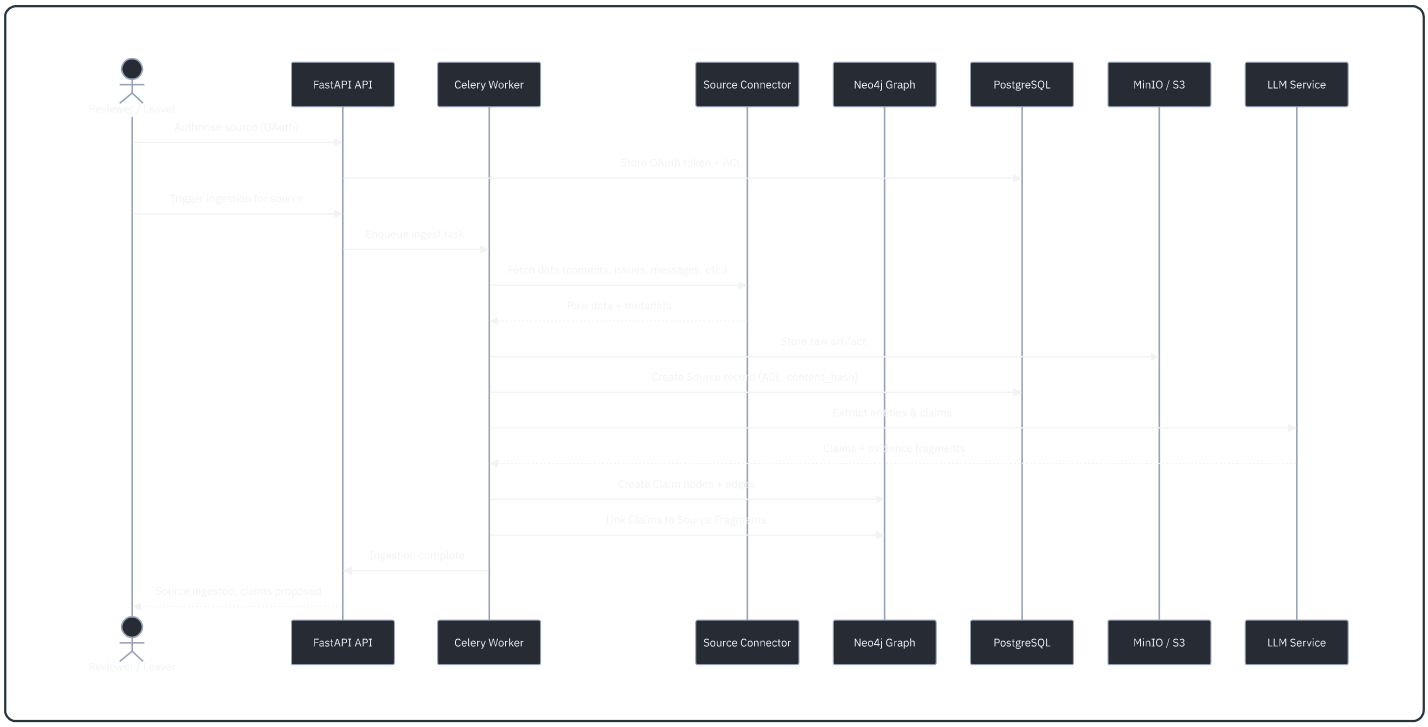

Imports, service calls, ticket links
Verify Layer
Link Claims to Source Fragments
Assign Confidence Scores 0.0 to 1.0
Detect Contradictions Conflicting sources
Flag Uncertainty Missing or low-confidence evidence
Review Layer - Humar
Reviewer Opens Handover Draft
Accept / Reject / Edit Claims


Merge Duplicate Nodes
Add Manual Notes
Approve Versioned Snapshot
Validate Layer - Automated Gate
Citation Integrity Check
Fail
Contradiction Re-scar
Confidence Recalculatior
Permission Re-validatior
Claim-level ACL check
Completeness Check
Ownership, blockers, next actions



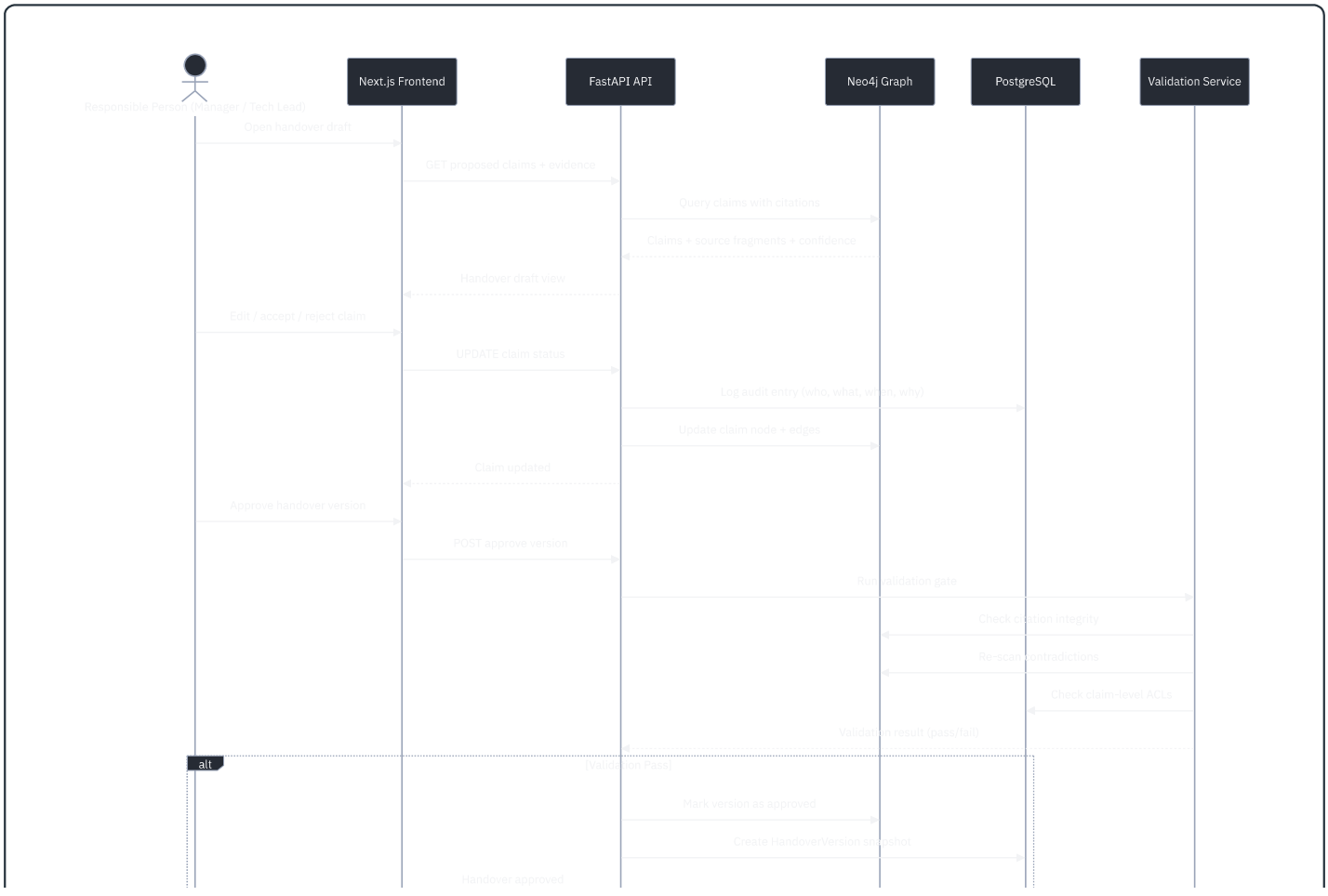
W3 **Sequence: ingesting a new source**

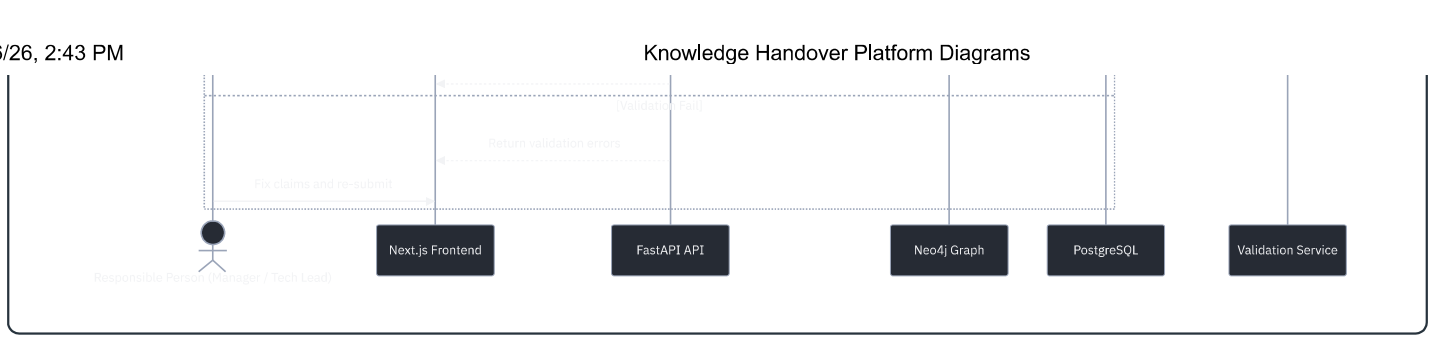
How a source is authorised, fetched, stored and turned into proposed claims.



W4 Sequence: review and validate a handover

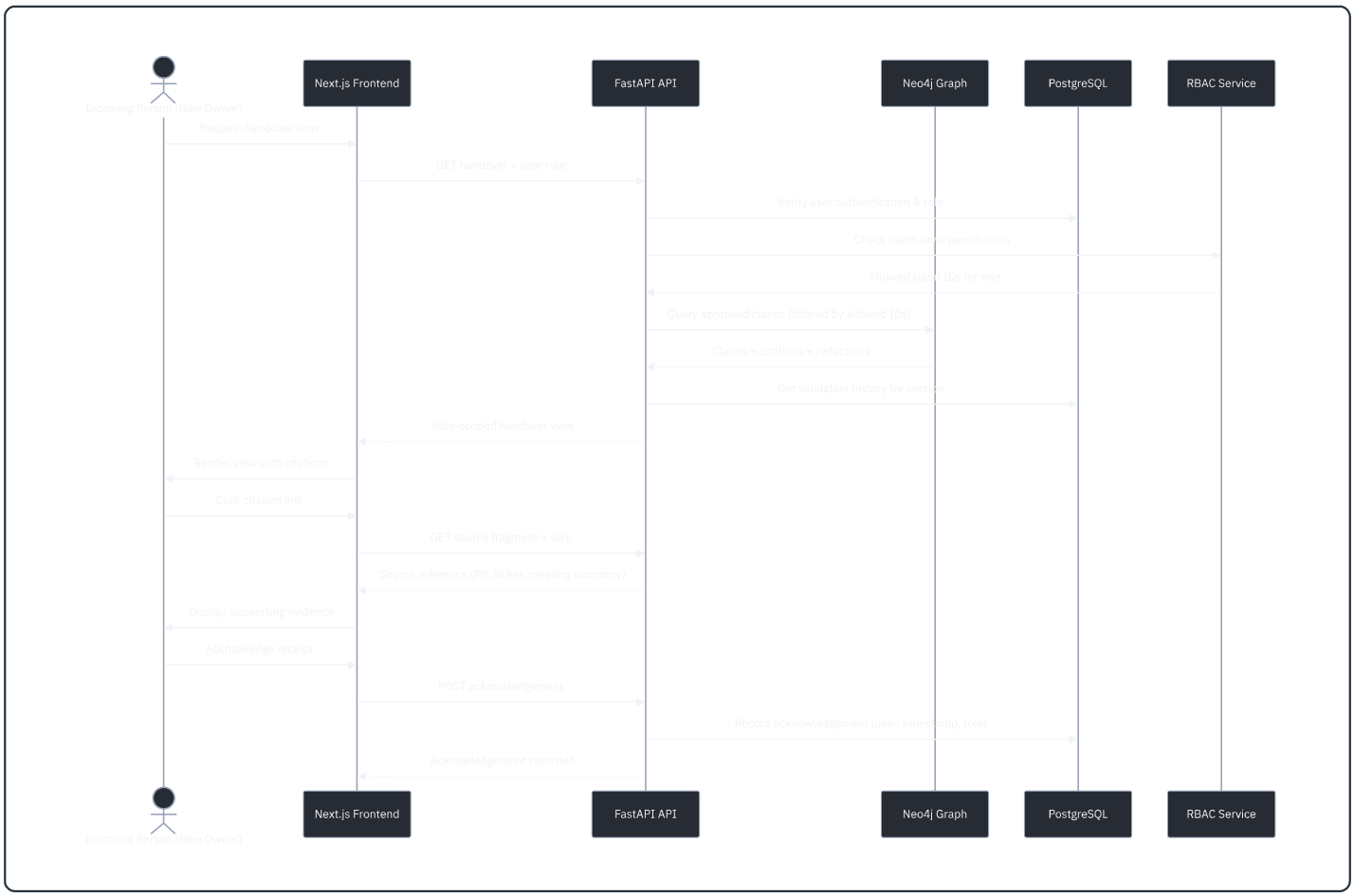
The reviewer edits claims, then approval runs the validation gate before a version is snapshotted.





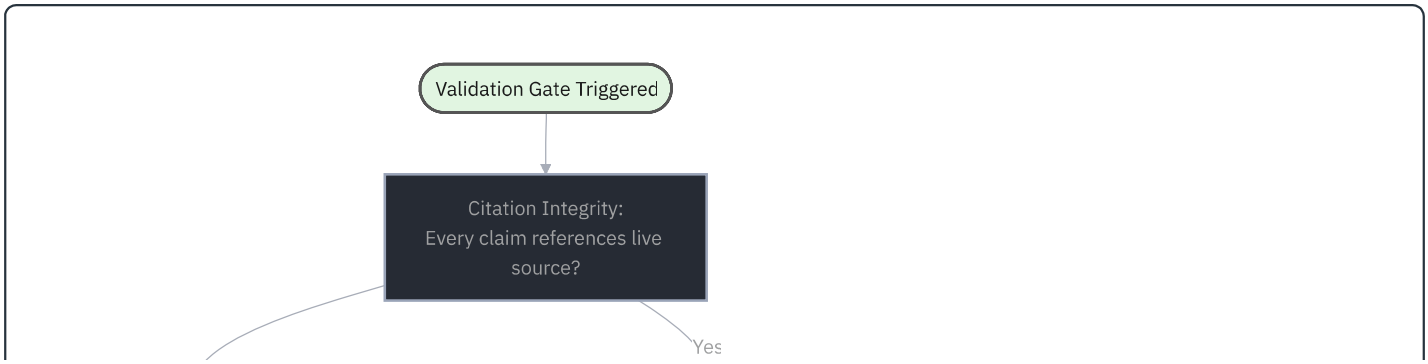
W5 Sequence: role-appropriate rendering

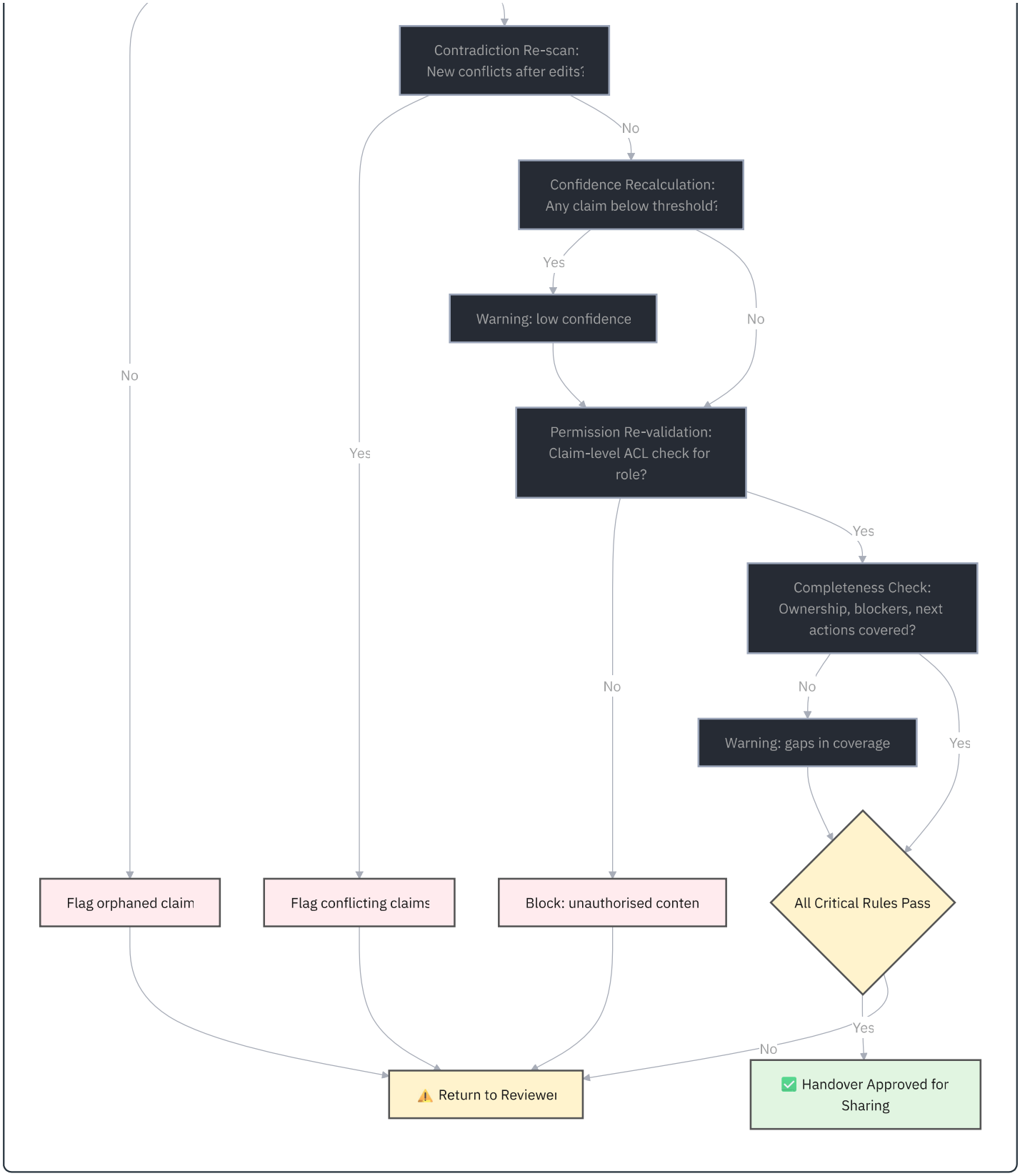
How the incoming owner sees only the claims their role may read, follows citations, and acknowledges receipt.



W6 Validation gate decision flow

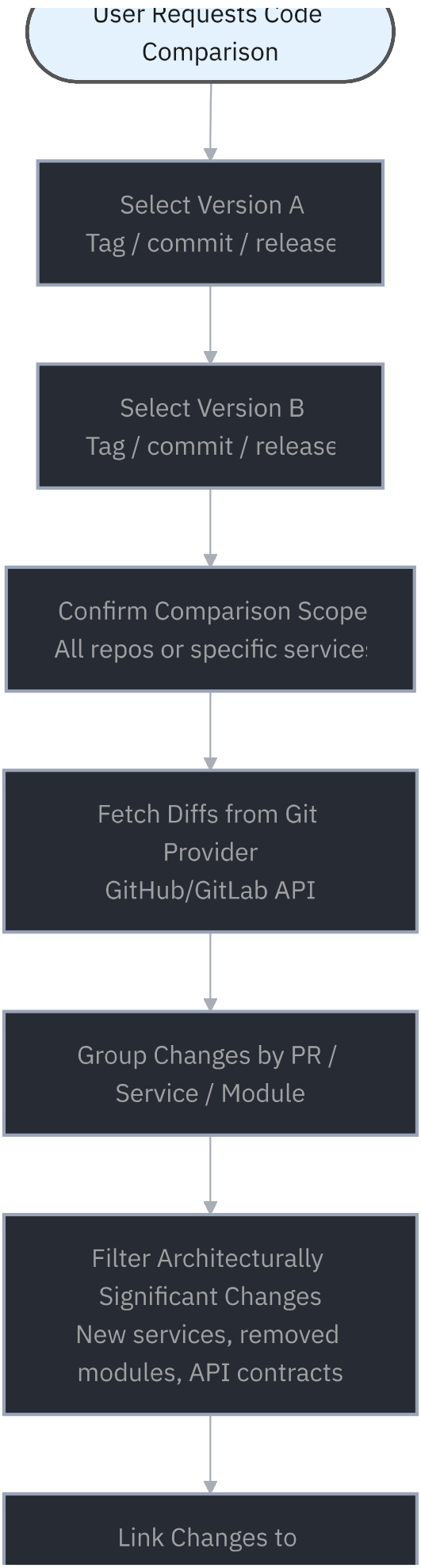
Five checks in order. Failures return to the reviewer; warnings continue to the next check.

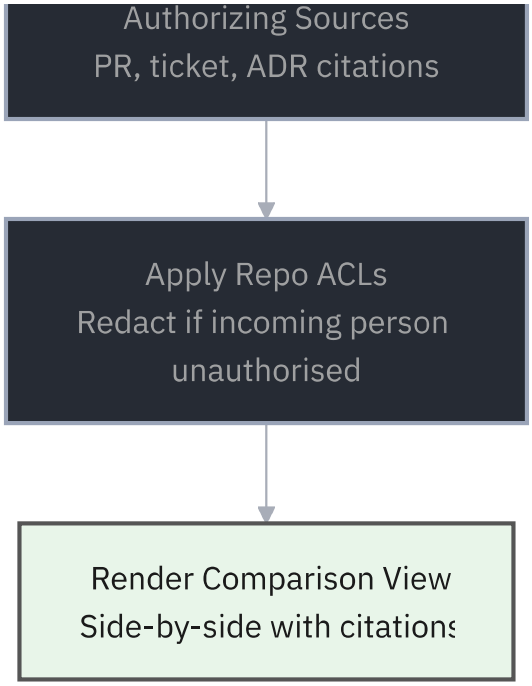




W7 Phase 3: on-demand code version comparison

Pick two versions, diff them, keep the architecturally significant changes, and cite what authorised each one.


User Requests Code Comparison
Select Version A
Tag / commit / release
Select Version B
Tag / commit / release
Confirm Comparison Scope
All repos or specific service
Fetch Diffs from Git Provider
GitHub/GitLab API
Group Changes by PR / Service / Module
Filter Architecturally Significant Changes
New services, removed modules, API contracts
Link Changes to



What changed from the source files

- `A5` Node IDs mixed `API_Pod` and `API_POD`. Mermaid treats these as different nodes, so the diagram showed duplicates. All use `API_POD` now.
- `A4` Relationship edges sat inside a second subgraph, which can pull nodes out of Node Types. They are now at the top level.
- `A2` The Graph Service node is renamed `GraphSvc` to avoid the `graph` keyword, and the two opposite edges to External Sources are one two-way edge.
- `W2` The quotes around "agreed" inside a label broke parsing, and the en dash in "0.0–1.0" is now "0.0 to 1.0". The gate is a decision diamond.
- `W6` The start node had no edge to the first check. The "gaps in coverage" warning looped back to the permission check; it now continues to the final rules check.
- `W7` The start node was not connected, and the S1, S2, S3 chain was declared twice. It is one chain from start to the comparison view now.
- `All` Every label is quoted, and coloured nodes have dark text so they stay readable in dark mode.